Project Overview

Project Title:

Smart Tourist Safety Monitoring & Incident Response System using AI, Geo-Fencing and Blockchain

Project Description:

A comprehensive, real-time tourist safety platform that integrates Artificial Intelligence for threat detection, Geo-Fencing technology for zone-based monitoring, and Blockchain for tamper-proof incident logging. The system provides authorities and tourists with proactive safety management tools.

Technology Stack

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Spring Boot (Java)
AI Module	Python (TensorFlow / OpenCV)
Blockchain	Ethereum / Hyperledger Fabric
Database	PostgreSQL + MongoDB
Geo-Fencing	Google Maps API / OpenLayers
Security	JWT Authentication + SSL
Deployment	Docker + AWS / Azure

DAY
01

📋 Planned Activity

Project Title Finalization

Jun-01 | Jun-01

✅ **Expected Deliverable**

Approved Project Title

What We Did

On Day 1, the team brainstormed and finalized the project title. We explored multiple AI and IoT-based ideas, evaluated feasibility, and chose a project that combines emerging technologies for real-world impact.

Activities Breakdown

- Brainstormed 10+ project ideas with team
- Evaluated each idea based on feasibility, innovation, and academic value
- Shortlisted 3 topics: Smart Traffic AI, Hospital Management System, Tourist Safety System
- Voted and finalized: 'Smart Tourist Safety Monitoring & Incident Response System using AI, Geo-Fencing and Blockchain'
- Got faculty approval for the title

Why This Topic?

Reason	Explanation
Real-World Impact	Tourist safety is a growing concern globally; AI can predict and prevent incidents
Technology Combination	Combines AI + Blockchain + Geo-Fencing — novel and challenging
Industry Relevance	Applicable to tourism boards, law enforcement, smart cities
Academic Rigor	Covers all required modules: DB, Frontend, Backend, AI, Security

DAY
02

📋 Planned Activity

Requirement Gathering

Jun-02 | Jun-02

✅ **Expected Deliverable**

Problem Statement

Problem Statement

Problem Statement:

Tourists visiting new locations are vulnerable to safety threats, emergencies, and natural disasters. Current systems lack real-time monitoring, tamper-proof logging, and intelligent zone-based alerts. There is no unified system that combines AI-based threat detection, Geo-Fencing, and Blockchain for tourist safety management.

Stakeholder Analysis

Stakeholder	Requirements
Tourists	Real-time safety alerts, SOS button, route guidance, emergency contacts
Tourism Authorities	Dashboard for monitoring tourist zones, incident reports, historical data
Police / Emergency Services	Instant notifications, GPS tracking, incident details
System Admin	Manage geo-zones, user accounts, blockchain records, AI model updates
Hotels / Resorts	Guest safety integration, emergency response coordination

Functional Requirements

- User registration, login, and profile management
- Real-time geo-fencing zone monitoring and alerts
- AI-based anomaly and threat detection from camera feeds
- SOS / emergency incident reporting by tourists
- Blockchain-based tamper-proof incident logging
- Admin dashboard for zone and user management
- Emergency service notification and coordination

Non-Functional Requirements

- System uptime: 99.9% availability
- Response time: < 2 seconds for alerts
- Data security: End-to-end encryption + JWT
- Scalability: Support 10,000+ concurrent users

DAY
03

📋 Planned Activity

Objective Definition

Jun-03 | Jun-03

✅ **Expected Deliverable**

Project Objectives

Project Objectives

Objective #	Objective Description
OBJ-01	Develop a real-time tourist safety monitoring platform using AI and IoT sensors
OBJ-02	Implement geo-fencing technology to define and monitor tourist safety zones
OBJ-03	Integrate AI-based anomaly detection for threat prediction from surveillance data
OBJ-04	Build blockchain-based immutable incident logging for transparency and accountability
OBJ-05	Create a mobile-responsive frontend for tourists with SOS and alert features
OBJ-06	Develop an authority dashboard for real-time monitoring and response coordination
OBJ-07	Implement secure REST APIs with JWT authentication for all system interactions
OBJ-08	Enable automated emergency notification to police and medical services

SMART Goals

Criterion	Description
Specific	Build a tourist safety system with AI, Geo-Fencing, and Blockchain in 30 days
Measurable	All 8 objectives must be completed with working demos
Achievable	Technologies are well-documented and team has Java/React/Python skills
Relevant	Directly addresses tourist safety gaps in smart city context
Time-Bound	Complete by June 30, 2025 with full submission

DAY
04

📋 Planned Activity

User & Module Identification

Jun-04 | Jun-04

✅ **Expected Deliverable**

Module List

User Roles

User Role	Description	Access Level
Tourist	End-user who uses the mobile app for safety features	Basic
Authority / Officer	Police, tourism officer who monitors the dashboard	Medium
System Admin	Manages users, zones, blockchain, AI models	Full
Emergency Responder	Receives alerts, updates incident status	Medium
Hotel Manager	Views guest safety data for their property zone	Limited

Module List

Module #	Module Name	Description
M-01	User Authentication	Registration, Login, JWT, Role-based access
M-02	Tourist Profile	Profile management, SOS contacts, travel history
M-03	Geo-Fencing Engine	Zone creation, boundary monitoring, breach alerts
M-04	AI Threat Detection	ML model for anomaly detection from sensors/cameras
M-05	Incident Management	Report, track, escalate incidents
M-06	Blockchain Ledger	Tamper-proof incident and transaction logging
M-07	Authority Dashboard	Real-time map, alerts, reports, analytics
M-08	Notification System	SMS, Email, Push notifications for alerts
M-09	Admin Panel	User management, zone config, system settings
M-10	Analytics & Reports	Incident trends, zone heatmaps, statistics

DAY
05

📋 Planned Activity

Use Case Diagram Preparation

Jun-05 | Jun-05

✅ **Expected Deliverable**

CRUD APIs

Use Case Summary

The following use cases have been identified for all actors in the system.

Actor	Use Case
Tourist	Register / Login
Tourist	View Safety Zones on Map
Tourist	Receive Geo-Fence Alerts
Tourist	Trigger SOS Emergency
Tourist	View Incident Status
Authority	Login to Dashboard
Authority	Monitor Live Tourist Locations
Authority	View and Respond to Incidents
Authority	Generate Incident Reports
Emergency Responder	Receive Automated Alert
Emergency Responder	Update Incident Response Status
Admin	Manage Users and Roles
Admin	Create and Edit Geo-Fencing Zones
Admin	View Blockchain Audit Logs
Admin	Configure AI Detection Parameters
System (AI)	Auto-detect Threats from Surveillance
System (Blockchain)	Auto-log All Incidents Immutably

CRUD API Matrix

Entity	Create	Read	Update	Delete
Tourist User	POST /api/users/register	GET /api/users/{id}	PUT /api/users/{id}	DELETE /api/users/{id}
Incident	POST /api/incidents	GET /api/incidents	PUT /api/incidents/{id}	DELETE /api/incidents/{id}

Geo-Zone	POST /api/zones	GET /api/zones	PUT /api/zones/{id}	DELETE /api/zones/{id}
Alert	POST /api/alerts	GET /api/alerts	PUT /api/alerts/{id}	—
Blockchain Log	POST /api/ledger	GET /api/ledger/{txId}	— (Immutable)	— (Immutable)

DAY
06

📋 Planned Activity

Database Requirement Analysis

Jun-06 | Jun-06

✅ **Expected Deliverable**

Table List

Database Tables Identified

Table Name	Purpose	Key Columns
users	Store all user accounts and roles	user_id, name, email, role, password_hash
tourist_profiles	Tourist-specific profile info	profile_id, user_id (FK), emergency_contact, current_location
geo_zones	Define geo-fencing zones	zone_id, zone_name, coordinates_json, risk_level, status
zone_breaches	Log zone entry/exit violations	breach_id, zone_id, user_id, breach_time, breach_type
incidents	Record all safety incidents	incident_id, type, location, severity, status, reported_by
incident_responses	Track response actions	response_id, incident_id, officer_id, action, timestamp
blockchain_logs	Immutable incident hash records	log_id, tx_hash, incident_id, block_number, timestamp
alerts	System-generated alerts	alert_id, type, message, recipient_id, sent_at, read_status
ai_detections	AI model detection results	detection_id, camera_id, threat_type, confidence, timestamp
notifications	Push/email notifications	notif_id, user_id, message, channel, sent_at

Database Design Decisions

- PostgreSQL: Used for structured relational data (users, incidents, zones)
- MongoDB: Used for unstructured AI detection logs and sensor data
- All tables follow 3NF (Third Normal Form) for minimal redundancy
- UUID used as primary keys for distributed system compatibility
- Timestamps stored in UTC format for consistency across time zones

DAY
07

📋 Planned Activity

ER Diagram Design

Jun-07 | Jun-07

✅ **Expected Deliverable**

ER Diagram

Entity-Relationship Diagram

The ER diagram shows all entities and their relationships. Core entities and their attributes are listed below.

ER Diagram: Smart Tourist Safety System — Full ER Diagram

🗃️ USERS	
Attribute	**Type / Description**
🔑 **user_id (PK)**	UUID, NOT NULL, UNIQUE
name	VARCHAR(100), NOT NULL
email	VARCHAR(150), UNIQUE, NOT NULL
password_hash	TEXT, NOT NULL
role	ENUM (tourist, authority, admin, responder)
created_at	TIMESTAMP, DEFAULT NOW()
is_active	BOOLEAN, DEFAULT TRUE

🔗 *One USER has one TOURIST_PROFILE (1:1)*
🔗 *One USER can report many INCIDENTS (1:N)*
🔗 *One USER receives many ALERTS (1:N)*

🗃️ TOURIST_PROFILES	
Attribute	**Type / Description**
🔑 **profile_id (PK)**	UUID, NOT NULL
🔗 user_id (FK → users)	UUID, NOT NULL
emergency_contact	VARCHAR(15)
nationality	VARCHAR(50)
current_lat	DECIMAL(10,8)
current_lng	DECIMAL(11,8)
last_updated	TIMESTAMP

🔗 *One TOURIST_PROFILE causes many ZONE_BREACHES (1:N)*

🗃️ GEO_ZONES	
Attribute	**Type / Description**
🔑 **zone_id (PK)**	UUID, NOT NULL

zone_name	VARCHAR(100), NOT NULL
coordinates_json	JSON (polygon boundary)
risk_level	ENUM (low, medium, high, critical)
center_lat	DECIMAL(10,8)
center_lng	DECIMAL(11,8)
radius_meters	INT
status	ENUM (active, inactive)

🔗 *One GEO_ZONE has many ZONE_BREACHES (1:N)*
🔗 *One GEO_ZONE has many INCIDENTS (1:N)*

🗃️ INCIDENTS

Attribute	Type / Description
🔑 **incident_id (PK)**	UUID, NOT NULL
🔗 reported_by (FK → users)	UUID
🔗 zone_id (FK → geo_zones)	UUID
incident_type	ENUM (SOS, threat, zone_breach, medical, accident)
severity	ENUM (low, medium, high, critical)
location_lat	DECIMAL(10,8)
location_lng	DECIMAL(11,8)
description	TEXT
status	ENUM (open, in_progress, resolved, closed)
created_at	TIMESTAMP

🔗 *One INCIDENT has many INCIDENT_RESPONSES (1:N)*
🔗 *One INCIDENT has one BLOCKCHAIN_LOG (1:1)*

🗃️ BLOCKCHAIN_LOGS

Attribute	Type / Description
🔑 **log_id (PK)**	UUID, NOT NULL
🔗 incident_id (FK → incidents)	UUID, UNIQUE
tx_hash	VARCHAR(66), UNIQUE — Ethereum TX hash
block_number	BIGINT
block_timestamp	TIMESTAMP
smart_contract_addr	VARCHAR(42)
data_hash	TEXT — SHA-256 of incident data

🔗 *BLOCKCHAIN_LOG is immutable (no UPDATE/DELETE)*

🗃️ AI_DETECTIONS

Attribute	Type / Description
🔑 **detection_id (PK)**	UUID, NOT NULL
camera_id	VARCHAR(50) — Source camera/sensor

threat_type	VARCHAR(100) — e.g., crowd surge, weapon
confidence_score	FLOAT — 0.0 to 1.0
detected_at	TIMESTAMP
location_lat	DECIMAL(10,8)
location_lng	DECIMAL(11,8)
🔗 incident_id (FK → incidents)	UUID, NULLABLE

🔗 *AI_DETECTION may trigger one INCIDENT (1:0..1)*

DAY
08

📋 Planned Activity

Database Schema Creation

Jun-08 | Jun-08

✅ **Expected Deliverable**

SQL Schema

SQL Schema Overview

The PostgreSQL schema was created with all tables, constraints, indexes, and foreign key relationships defined below.

ER Diagram: Database Schema Relationships

🗃️ Schema: Relationships Map	
Attribute	**Type / Description**
users → tourist_profiles	1:1 via user_id
users → incidents	1:N via reported_by
users → alerts	1:N via recipient_id
geo_zones → zone_breaches	1:N via zone_id
geo_zones → incidents	1:N via zone_id
incidents → incident_responses	1:N via incident_id
incidents → blockchain_logs	1:1 via incident_id
incidents → ai_detections	1:N via incident_id
users → incident_responses	1:N via officer_id

Index Strategy

Table	Index Column(s)	Reason
users	email	Fast login lookup
incidents	zone_id, status, created_at	Dashboard filtering and sorting
geo_zones	status, risk_level	Active zone queries
ai_detections	detected_at, threat_type	Time-series analysis
blockchain_logs	tx_hash	Quick hash verification

DAY
09

📋 Planned Activity

UI Wireframe Design

Jun-09 | Jun-09

✅ **Expected Deliverable**

Page Layouts

Wireframe Page List

Page #	Page Name	User Role	Key Elements
P-01	Landing / Home	All	Hero section, login/register CTA, feature highlights
P-02	Tourist Registration	Tourist	Form: name, email, phone, emergency contact, nationality
P-03	Tourist Login	Tourist	Email/password form, forgot password link
P-04	Tourist Dashboard	Tourist	Map view, current zone status, SOS button, alerts feed
P-05	Authority Login	Authority	Secure login with 2FA option
P-06	Authority Dashboard	Authority	Live map, incident list, alert bell, statistics cards
P-07	Incident Detail View	Authority	Incident info, location, timeline, response actions
P-08	Geo-Zone Management	Admin	Map with zone polygons, create/edit/delete zones
P-09	User Management	Admin	Table of users, roles, activation toggle, search/filter
P-10	Blockchain Audit Log	Admin	Table of TX hashes, block numbers, timestamps, verify button
P-11	Analytics Dashboard	Authority/Admin	Charts: incidents/week, zone heatmap, AI alert trends
P-12	Notification Center	All	Push notification history, read/unread status
P-13	SOS Emergency Page	Tourist	Big red SOS button, GPS share, emergency contacts list

UI Design Principles

- Mobile-first responsive design (320px to 1920px breakpoints)
- Color coding: Red = danger/SOS, Green = safe zone, Yellow = caution, Blue = info
- Map-centric UI: Tourist and authority views center around interactive maps
- Accessibility: WCAG 2.1 AA compliant — high contrast, keyboard navigable
- React.js with Tailwind CSS for rapid, consistent UI development

DAY
10

📋 Planned Activity

Login & Dashboard UI Design

Jun-10 | Jun-10

✅ **Expected Deliverable**

UI Screens

Login Screen Design

Component	Description
Logo & Title	App logo, 'Tourist Safety System' header in navy blue
Email Input	Placeholder: 'Enter your email', validation for proper format
Password Input	Masked input with show/hide toggle
Role Selector	Dropdown: Tourist / Authority / Admin
Login Button	Primary blue button, disabled until form is valid
Forgot Password	Link below button, redirects to reset flow
Register Link	For tourists: 'New here? Register now' link
Error Messages	Red inline validation messages

Tourist Dashboard UI Components

Component	Description
Header Bar	App title, user avatar, notification bell, logout icon
Map View (Main)	Fullscreen interactive map with tourist's live location pin
Zone Status Badge	Color-coded badge: SAFE (green) / CAUTION (yellow) / DANGER (red)
SOS Button	Large red floating button — bottom right, always visible
Alert Feed	Scrollable list of recent alerts with timestamps
My Location Card	GPS coordinates, zone name, last updated time
Emergency Contacts	Quick dial buttons for preset emergency numbers
Settings Icon	Top right corner for profile and preferences

Authority Dashboard UI Components

- Live map showing all tourists as blue dots within each zone
- Incident panel on right side: sortable by severity, time, type
- Statistics bar at top: Total tourists tracked, Active incidents, Zone alerts today
- Zone overlay layer: Color-coded polygons with risk levels
- Alert notification popup: Auto-appears for critical incidents

DAY
11

📋 Planned Activity

Navigation & Form Design

Jun-11 | Jun-11

✅ **Expected Deliverable**

UI Prototype

Navigation Structure

Navigation Item	Tourist	Authority	Admin
Home / Dashboard	✅	✅	✅
My Safety Map	✅	❌	❌
SOS / Emergency	✅	❌	❌
Live Monitor Map	❌	✅	✅
Incidents	View only	✅ Full Access	✅ Full Access
Geo-Zone Manager	❌	View only	✅ Full CRUD
User Manager	❌	❌	✅
Blockchain Logs	❌	View	✅
Analytics	❌	✅	✅
Profile / Settings	✅	✅	✅

Key Forms Designed

Form Name	Fields	Validation
Tourist Registration	Name, Email, Phone, Password, Emergency Contact, Nationality	Email format, phone 10 digits, password strength
Incident Report Form	Type (dropdown), Severity, Description, Location (auto-GPS), Photo upload	Required fields, max 500 chars description
Geo-Zone Creator	Zone Name, Risk Level, Draw on Map (polygon), Activate toggle	Polygon must be closed, name required
SOS Trigger Form	Message (optional), Location (auto), Share contacts toggle	Instant submission, no validation delays
AI Config Form	Model type, Confidence threshold (0.0-1.0), Camera IDs, Alert delay	Confidence must be 0.5-0.99 range

Prototype Flow

- Figma/InVision prototype linked: All 13 screens connected with click-through interactions
- Tourist flow: Landing → Register → Dashboard → View Map → Trigger SOS
- Authority flow: Login → Dashboard → View Alert → Open Incident → Take Action
- Admin flow: Login → Admin Panel → Manage Zones → View Blockchain Logs

DAY
12

📋 Planned Activity

Design Review

Jun-12 | Jun-12

✅ **Expected Deliverable**

Design Approval

Design Review Checklist

Review Area	Status	Comments
Project Title	✅ Approved	Title is clear and relevant
Problem Statement	✅ Approved	Well-defined scope
Objectives (8)	✅ Approved	All SMART objectives met
Module List (10)	✅ Approved	All modules accounted for
Use Cases	✅ Approved	Covers all actors and interactions
CRUD APIs	✅ Approved	REST conventions followed
Database Tables (10)	✅ Approved	3NF normalized design
ER Diagram	✅ Approved	All relationships correctly mapped
SQL Schema	✅ Approved	Indexes and constraints defined
UI Wireframes	✅ Approved	13 screens fully designed
UI Screens	✅ Approved	Login + Dashboard components complete
UI Prototype	⚠️ Minor Fix	SOS page needs larger button on mobile
Overall Design	✅ Approved	Ready to proceed to development

Faculty Review Feedback

Faculty Comments:

1. Strong project with real-world applicability. Blockchain integration is innovative.
2. Ensure AI module has fallback for low-confidence detections to avoid false positives.
3. SOS button must work offline (cached last known location). Add to backlog.
4. **Overall: Design phase is well-executed. Approved for development phase.**

DAY
13

📋 Planned Activity

Frontend Environment Setup

Jun-13 | Jun-13

✅ **Expected Deliverable**

React Project Setup

Frontend Environment Setup

Tool / Package	Version	Purpose
Node.js	v20.x LTS	JavaScript runtime environment
npm	v10.x	Package manager
React.js	v18.2	Frontend framework
Vite	v5.x	Build tool (faster than CRA)
Tailwind CSS	v3.4	Utility-first CSS framework
React Router DOM	v6.x	Client-side routing and navigation
Axios	v1.6	HTTP client for REST API calls
Leaflet.js / React-Leaflet	v4.x	Interactive map integration
JWT Decode	v4.x	Client-side JWT token parsing
React Toastify	v10.x	Toast notification system
Chart.js / Recharts	v3.x	Analytics and data visualization
ESLint + Prettier	Latest	Code quality and formatting

Project Folder Structure

```
tourist-safety-frontend/
├── src/
│   ├── components/   # Reusable UI components
│   ├── pages/        # Route-level page components
│   ├── services/     # Axios API call functions
│   ├── context/      # React Context (Auth, Alerts)
│   ├── hooks/        # Custom React hooks
│   ├── utils/        # Helper functions, validators
│   └── App.jsx       # Root component with routes
├── public/           # Static assets (favicon, index.html)
└── package.json      # Dependencies manifest
```

Environment Variables (.env)

- VITE_API_BASE_URL=http://localhost:8080/api
- VITE_MAPS_API_KEY=[Google Maps / OpenLayers API Key]
- VITE_BLOCKCHAIN_RPC=http://localhost:8545
- VITE_WS_URL=ws://localhost:8080/ws (WebSocket for real-time alerts)

Days 1–13 Summary

Day	Date	Planned Activity	Expected Deliverable
01	Jun-01	Project Title Finalization	Approved Project Title
02	Jun-02	Requirement Gathering	Problem Statement
03	Jun-03	Objective Definition	Project Objectives
04	Jun-04	User & Module Identification	Module List
05	Jun-05	Use Case Diagram Preparation	CRUD APIs
06	Jun-06	Database Requirement Analysis	Table List
07	Jun-07	ER Diagram Design	ER Diagram
08	Jun-08	Database Schema Creation	SQL Schema
09	Jun-09	UI Wireframe Design	Page Layouts
10	Jun-10	Login & Dashboard UI Design	UI Screens
11	Jun-11	Navigation & Form Design	UI Prototype
12	Jun-12	Design Review	Design Approval
13	Jun-13	Frontend Environment Setup	React Project Setup